

局域网 IM 与文件共享项目技术选型研究报告

执行摘要

结论先行：如果这是一个今天才启动的、桌面优先、跨平台、局域网 IM + 文件共享项目，我不建议再从 Qt5 起步。更稳妥的主线是：**Qt 6.8 LTS + C++17 + CMake/Ninja + Widgets 优先、少量 QML 可选；MVP 先做纯 C++ 单进程客户端**，在代码边界上预留日后拆分后台服务的能力；**暂不推荐 Tauri 作为首发主路线，也不建议在 LGPL 路线下直接采用 Qt GRPC**。核心原因很简单：Qt 5.15 的标准支持已在 **2025-05-26** 结束，最后一个 5.15 补丁为 **5.15.19**；而 **Qt 6.8 LTS** 的标准支持到 **2029-10-08**，且 Qt 6 的官方平台矩阵已经覆盖 Windows x64/ARM64、macOS Intel/Apple Silicon、Linux x86_64/ARM64。对一个要活三到五年的新项目，这个差距是决定性的。¹

更具体地说：

决策项	最终建议
Qt5 还是 Qt6	Qt 6.8 LTS 。不要新开 Qt5。 ²
纯 C++ 还是混合架构	MVP 先纯 C++/Qt ；当文件索引、传输、加密、增量同步复杂度上来后，再把后台拆成独立 Rust 服务进程。 ³
Rust / Go / Tauri	优先 Rust 备选，不优先 Go，不推荐 Tauri 作为首发 UI 主线 。Tauri 在 Linux 上依赖 WebKitGTK 4.1、GTK3，使用托盘还会额外依赖 appindicator，这对 openKylin / UKUI / LoongArch 的首发兼容面并不友好。 ⁴
UI 技术	Qt Widgets 优先 ；只在欢迎页、设置页、文件卡片等局部使用 QML。Qt 6 的 QML 工具链更强，但经典桌面 IM/托盘/传输面板这类 UI，Widgets 的实现速度和稳定性更高。 ⁵
是否引入 Qt WebEngine	MVP 不引入 。它会显著放大编译、打包、沙箱与多架构压力；如果只是做设置页、帮助页、登录页，不值得。Qt WebEngine 还带来 Chromium 级依赖，并在源码构建时要求 C++20。 ⁶
IPC 方案	纯 C++ 单进程下 不需要 IPC ；若拆后台，优先 QLocalSocket/QLocalServer 或 WebSocket/本地 HTTP 。如果你走开源 LGPL Qt 且想闭源分发， 不要把 Qt GRPC 当成默认 IPC 方案 ，因为 Qt 文档明确列出 Qt GRPC 在开源路线下属于 GPLv3 模块 。 ⁷

基于以上证据，我给出的一句话推荐是：**用 Qt 6.8 LTS 做一个 Widgets-first 的原生桌面客户端，MVP 不上 WebEngine、不上 Tauri、不做双语言双进程；等 MVP 验证通过后，再把“传输引擎 / 索引服务 / 加密与同步”下沉为 Rust 独立进程**。这条路线对个人开发者的时间成本、平台兼容性、后续演进空间三者的平衡最好。⁸

项目假设与决策原则

本报告按以下假设展开：你是**中文环境的个人开发者**；目标产品是**桌面优先**的局域网 IM + 文件共享工具；首发平台为 **Windows、macOS、Linux、openKylin/麒麟系**；Android/iOS 不作为当前高优先级；MVP 目标周期

按十二到十六周估算；分发模型默认为可商业化、可能闭源。这些假设决定了技术选型必须优先考虑：长期支持、托盘与后台驻留、局域网发现、文件传输稳定性、打包可控性、以及麒麟系 Linux 的实际可落地性。

因此，本报告的判断标准不是“哪套技术理论上更先进”，而是以下五件事：能否最快做出可用桌面软件、能否跨 Windows/macOS/Linux 真正稳定运行、能否在 openKylin/UKUI/LoongArch 这类边缘组合下仍有可行路径、能否在闭源商业分发下规避许可踩坑、以及后续是否容易拆分后台服务做第二阶段演进。这一标准会天然让“技术最潮”的方案失分，也会让“工程中庸但稳定”的方案得分更高。

Qt5、Qt6 与替代路线对比

从官方支持周期看，Qt5 对新项目已经不合适。Qt 官方博客明确说明 **Qt 5.15 的标准支持在 2025-05-26 结束，5.15.19** 是最后一个 5.15 发布版；继续使用需要 Extended Security Maintenance 或其他附加服务。相比之下，Qt 官方发布页显示 **Qt 6.8 LTS** 仍在标准支持期内，直到 **2029-10-08**；而 **Qt 6.11** 是当前最新非 LTS 主线。对一个 2026 年才立项的应用，继续押 Qt5，等于一启动就落在“安全更新与生态迁移都在往后推”的旧平台上。⁹

Qt 6 还有一个现实优势：**平台矩阵更现代**。Qt 6 官方支持页列出了 **Windows x86_64 与 Windows on ARM64、macOS x86_64 / x86_64h / arm64**，以及较新的 Ubuntu、Debian、RHEL、openSUSE 发行版；而 Qt 5.15 的支持矩阵仍停留在 **Windows 7/8.1/10、macOS 10.13/10.14/10.15 且仅 x86_64 / x86_64h、以及 Ubuntu 18.04** 这一代。对你关心的 Apple Silicon 与现代 Linux 组合，Qt 6 明显更对路。¹⁰

Qt5 与 Qt6 核心对比表

下表综合了 Qt 官方文档、Qt 官方博客与当前平台支持页面；涉及支持周期、平台矩阵、构建体系、许可与模块状态的条目均优先引用原始文档。¹¹

维度	Qt5.15	Qt6 当前情况	对本项目的判断
C++ 绑定	标准绑定为 C++；Qt 5.15 已支持 CMake 的无版本目标，但历史上以 qmake 为主。 ¹²	标准绑定为 C++；Qt 自身构建系统已迁到 CMake，qmake 仍可用于应用，但整体生态是 CMake-first。 ¹³	新项目应直接 CMake 化 ，避免先写 qmake 再迁移。
Windows 支持	支持 Windows 7/8.1/10 的 x86/x64。 ¹⁴	支持 Windows 10/11 x86_64，以及 Windows on ARM 的 ARM64；Qt 6.12 将是支持 Windows 10 的最后一版。 ¹⁵	若你希望覆盖 Windows ARM64 或更长期的 Windows 主线， Qt6 更合适 。
macOS 支持	官方矩阵仅列 x86_64 / x86_64h 与较老 macOS。 ¹⁴	官方矩阵覆盖现代 macOS 与 arm64。 ¹⁶	Apple Silicon 直接把天平推向 Qt6 。
Linux 发行版支持	官方矩阵偏旧，主要是 Ubuntu 18.04、RHEL 7、Generic Linux x86/x64。 ¹⁴	官方矩阵覆盖 Ubuntu 22.04 / 24.04、Debian 11.6 / 12、RHEL 8/9/10、openSUSE 15.6 / 16.0；Linux ARM64 也进入支持范围，但桌面 ARM 仍以 Raspberry Pi 5 + Ubuntu 24.04 为参考。 ¹⁷	对现代 Linux 与 ARM64， Qt6 明显更现实 。

维度	Qt5.15	Qt6 当前情况	对本项目的判断
openKylin / 麒麟兼容	Qt 官方并未单列 openKylin/麒麟；未列出的平台不属于官方支持配置。 ¹⁸	同左；但 openKylin 官方已提供 AMD64 / ARM / LoongArch 镜像，并采用 DEB/apt/dpkg。 ¹⁹	这意味着麒麟系应被视为 可运行但需自保 的平台，且 Qt6 + 原生 DEB 更符合当前生态。
安装包与部署	有 windeployqt / macdeployqt / Linux 手工部署路径，但工具链与文档已进入存量维护阶段。Linux 平台部署仍需自己管理 libqxcb.so 等插件。 ²⁰	除部署工具外，Qt6 也与 CMake/CPack、Qt IFW 更顺；CPack 支持 DEB/RPM/NSIS/WiX/DragNDrop/IFW/ApplImage，新版还加入 ApplImage 生成器。Qt IFW 明确支持 Windows/macOS/Linux。 ²¹	Qt6 + CMake + CPack/IFW 是更现代的打包基线。
WebView / HTML 集成	可用，但你实际面对的是旧版 QtWebEngine 路线；对新项目价值不大。	Qt WebEngine 仍是 Chromium 路线，支持 Win/Linux/macOS；Qt WebView 走原生 WebView，Windows 可用 WebView2。Qt WebEngine 源码构建要求 C++20，且 Linux 上有沙箱/内核约束。 ²²	MVP 不建议引入 WebEngine ；除非你明确要做富 HTML 管理后台。
QML 支持	QML/Qt Quick 成熟，但许多性能与图形后端仍处在 OpenGL/ANGLE 时代。Qt5 文档明确提到 Qt Quick 2 依赖 OpenGL ES 2.0、DirectX 9/11 via ANGLE 或其他渲染器。 ¹⁴	Qt6 引入 RHI，Qt Quick 可落到 D3D11、Vulkan、Metal、OpenGL；QML type compiler/script compiler 可提升启动与执行效率。 ²³	如果你未来想把 UI 做得更现代， Qt6 的 QML 更值得投资 。
性能	Widgets-first 工具型软件可用；但 Qt Quick 图形路径更旧。	Qt6 在 Qt Quick 上的主要提升来自 RHI + QML 编译链 ，不是“所有应用无脑更快”。 ²⁴	对本项目这类桌面工具， 代际性能不是第一矛盾；支持周期与平台兼容更重要 。
内存占用	官方没有给出一组适用于 IM 工具的统一基准。	官方同样没有给出统一桌面基准；但可以确定：一旦引入 Qt WebEngine，内存与包体往往会被 Chromium 主导，而不是被 Qt5/6 差异主导。QML 编译链更像是启动与执行效率优化。 ²⁵	是否引入 WebEngine，比 Qt5/Qt6 本身更影响内存 。
构建工具链	Qt5 的构建体系建立在 qmake 之上；Qt 5.15 才提供更好的一致化 CMake API。 ²⁶	Qt6 自身已迁到 CMake；源码构建依赖现代 CMake/Ninja；跨编译也转向 toolchain file；自定义插件/内部库不再建议走 qmake。 ²⁷	新项目直接用 CMake/Ninja，不要犹豫 。

维度	Qt5.15	Qt6 当前情况	对本项目的判断
许可	Qt 可在商业许可下使用，也可在 LGPLv3 下使用；但一部分模块并非 LGPL。 ²⁸	同左，而且 Qt 官方明确列出： Qt GRPC、Qt HTTP Server、Qt MQTT、Qt Network Authorization、Qt Virtual Keyboard、Qt Wayland Compositor 等开源路线下是 GPLv3 模块。 ²⁸	若你计划闭源商业化且只用开源版 Qt， 不要默认用 Qt GRPC。
长期维护与社区趋势	进入收尾阶段。	Qt6 是活跃主线。Krita 已把 Qt6 端口作为 Krita 6，并明确提到多家 Linux 发行版正在放弃 Qt5；OBS 也已规划并完成向 Qt6 迁移。Telegram Desktop 持续以 Qt 为基础。 ²⁹	技术趋势站在 Qt6 一边。
安全修复与 LTS	Qt 5.15 标准支持已结束，只剩附加服务。 ³⁰	Qt 6.8 LTS 仍在标准支持期；Qt 6.5 LTS 已于 2026-04-03 后进入 EoS。 ³¹	如果现在开工， 应选 Qt 6.8 LTS，而不是 6.5 LTS，更不是 Qt5。

纯 C++、Qt 加 Rust/Go、Tauri 的取舍

对你的项目，真正需要比较的不是“能不能做”，而是**用一个人能不能做完、做稳、做得可打包**。我的判断如下。涉及 Tauri、IPC 与 gRPC 语言支持的客观事实，下面均引官方文档。³²

方案	适合做什么	优点	短板	结论
纯 C++ Qt 单进程	MVP、桌面首发、经典 IM/托盘/文件传输面板	一个语言、一个构建系统、一个调试链；最小认知负担；托盘、窗口、文件系统、网络 API 都在 Qt 里。 ³³	后台逻辑、传输引擎、UI 可能耦合；后续拆服务要补架构债。	MVP 最优
Qt 前端 + Rust 后台服务	第二阶段，当你需要更强的传输引擎、索引器、同步器、加密与崩溃隔离	可以把复杂后台隔离成独立进程；未来更容易做无 UI service。gRPC/本地 socket 都可行，Rust 也在 gRPC 官方语言列表中。 ³⁴	双语言、双工具链、双调试流程；个人开发者前期速度会下降。	中期优选
Qt 前端 + Go 后台服务	后台协议服务、扫描器、CLI、守护进程	gRPC Go 成熟，上手快。 ³⁵	UI 与后台语义割裂；进程边界清晰但整体包体、调试与分发复杂度并不低。	可行，但不如 Rust 路线长期清晰
Tauri + Rust	Web 技能极强、UI 主要是网页、且目标平台不是 openKylin/UKUI/LoongArch 优先时	Tauri 官方强调“tiny and fast binaries”，并采用多进程模型。 ³⁶	Linux 依赖 webkit2gtk 4.1 、GTK3，托盘还要 appindicator；这会把 UKUI/麒麟系、LoongArch 的首发兼容风险抬高。 ³⁷	不建议作为本项目首发主线

对你这个项目，我的建议不是“永远别用 Rust/Go”，而是：**先别在 MVP 阶段同时背上 Qt + Rust/Go + IPC + 双打包链的全部复杂度**。先把产品节奏跑起来，等你发现“文件索引器 / 增量同步 / 后台常驻服务”确实成了瓶颈，再拆后台，收益才真正大于成本。

Linux、麒麟与多架构兼容性

Qt 官方支持页明确提醒：**凡是没有列入支持矩阵的配置，都不属于 Qt Project 的官方支持平台**，即使“可能能跑”。这对于 openKylin / 麒麟、UKUI、LoongArch 尤其重要：它们**不是不能做**，但更接近“自维护兼容平台”，而不是“开箱即用官方保障平台”。与此同时，openKylin 官方下载页已经明确给出 **AMD64、ARM、LoongArch** 镜像；其打包文档也明确说明 openKylin 以 **DEB + apt/dpkg** 为底座。综合来看，麒麟系的合理策略不是“追求 Qt 官方预编译包一把过”，而是**围绕 DEB 原生包、自己做 CI 与兼容回归**。³⁸

另一个必须提前看到的现实是：Qt 6 官方 Linux ARM 桌面支持虽然已经存在，但文档同时写明它以 **Raspberry Pi 5 + Ubuntu 24.04** 为参考平台，且 **Qt Online Installer 的官方二进制是在 Ubuntu 24.04、glibc 2.39 上构建**的；如果目标系统 glibc 更旧，就需要**自行从源码重建**。这意味着你若想覆盖 openKylin/麒麟、ARM64、甚至 LoongArch，“**依赖官方在线安装器预编译包**”不是长期策略；更稳妥的是采用**原生发行版包或自建源码产物**。¹⁷

Linux 与 openKylin 的重点坑位

坑位	风险说明	建议
图形栈与 libGL / Mesa	Qt 官方 Linux 文档明确要求 OpenGL 库与头文件；X11/xcb 路线还依赖一串 xcb 相关库。 ³⁹	Widgets-first ；Linux 包内显式校验平台插件与图形库；CI 在最小桌面镜像上做冷启动测试。
X11 与 Wayland 差异	Qt 文档明确指出：在 Wayland 下，客户端通常 不能手动设置顶层窗口位置 ；很多“强行弹出到屏幕某坐标”的桌面习惯做法会失效。 ⁴⁰	传输提示、登录弹窗、消息提醒不要依赖绝对坐标；优先使用系统通知与正常窗口激活。
系统托盘	<code>QSystemTrayIcon</code> 在 Linux 上依赖桌面环境实现 StatusNotifierItem 或 XEmbed；GNOME Shell 3.26 起部分激活语义还需要扩展。 ⁴¹	把“ 无托盘回退路径 ”列为需求：没有托盘时，必须仍能通过主窗口、菜单、启动项、通知入口操作。UKUI 要单独做实机验证。
Qt WebEngine 沙箱	Qt WebEngine 在 Linux 上依赖内核 user namespaces 与 seccomp-bpf ；Qt 文档还特别点名 Debian/Ubuntu 衍生系常见的 <code>unprivileged_usersns_clone</code> 问题。 ⁴²	MVP 不引入 Qt WebEngine 。如果未来一定要用，先在 openKylin/麒麟实机验证沙箱行为。
WebKitGTK 依赖	这不是 Qt 的坑，而是 Tauri/Linux WebView 路线的坑：Tauri v2 在 Linux 依赖 <code>webkit2gtk-4.1</code> 、GTK3，系统托盘还会带入 <code>appindicator</code> 依赖。 ³⁷	这正是我不推荐 Tauri 作为你首发主线的重要原因之一。
glibc 基线	ApplImage 官方文档明确提醒：为了兼容旧系统，应在你要支持的最老 基础系统 上构建，否则容易出现 GLIBC 版本不兼容。 ⁴³	Linux x86_64 / arm64 的 ApplImage 建议在 Ubuntu 22.04 或 Debian 12 这类保守基线构建；openKylin 优先发 DEB 。

多架构与交叉编译建议

Qt 官方文档对“交叉编译 Qt”说得很清楚：你需要**工具链 + sysroot**，而且在 Qt 6 里，交叉编译目标版 Qt 之前，主机上必须先有**同版本 host Qt**。从流程复杂度看，这更适合“你真的有某个目标设备 SDK / sysroot”的场景，而不是桌面发行版全家桶兼容。对桌面产品而言，我更建议用**原生构建优先、交叉编译为辅**。⁴⁴

我的推荐矩阵如下：

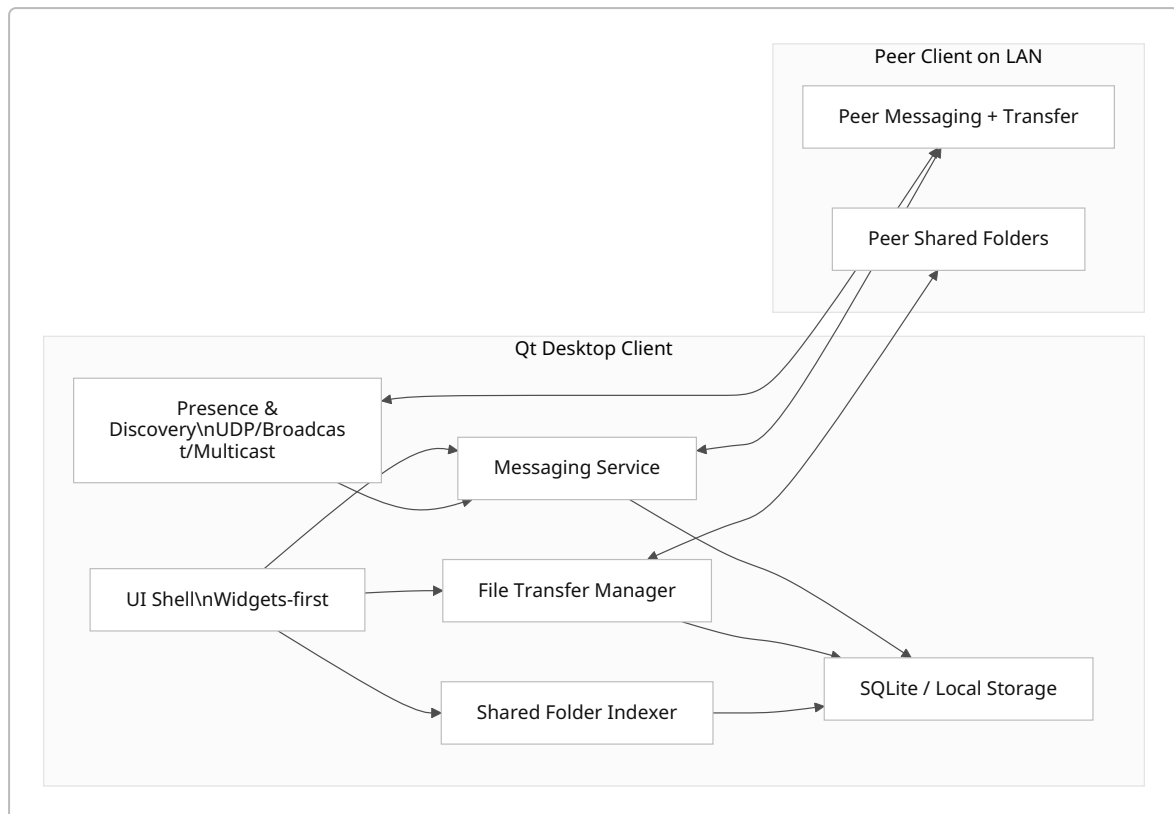
架构	可行性	推荐做法
x86_64	最高	Windows/macOS/Linux 都走原生 runner；这是主发布线。
ARM64	高	Windows ARM64 走 MSVC 原生；Linux ARM64 如果目标是 Ubuntu 24.04 / Debian 12，优先原生 runner 或官方支持环境。 ⁴⁵
LoongArch	可行， 但自维护	Qt 6.5 起已有 <code>Q_PROCESSOR_LOONGARCH</code> 宏，openKylin 也有 LoongArch 镜像，但 Qt 官方支持矩阵 未列 LoongArch 桌面 。因此建议采用 自建 LoongArch 原生构建节点 ，不要把 LoongArch 版首发建立在 WebEngine、Tauri 或复杂 HTML UI 上。 ⁴⁶

因此，对 Linux/openKylin/麒麟 的真实建议是：
x86_64 与 ARM64 用容器化基线 + 原生 runner；LoongArch 用自托管原生节点；MVP 放弃 WebEngine；首发包型以 DEB 为主，ApplImage 为辅。

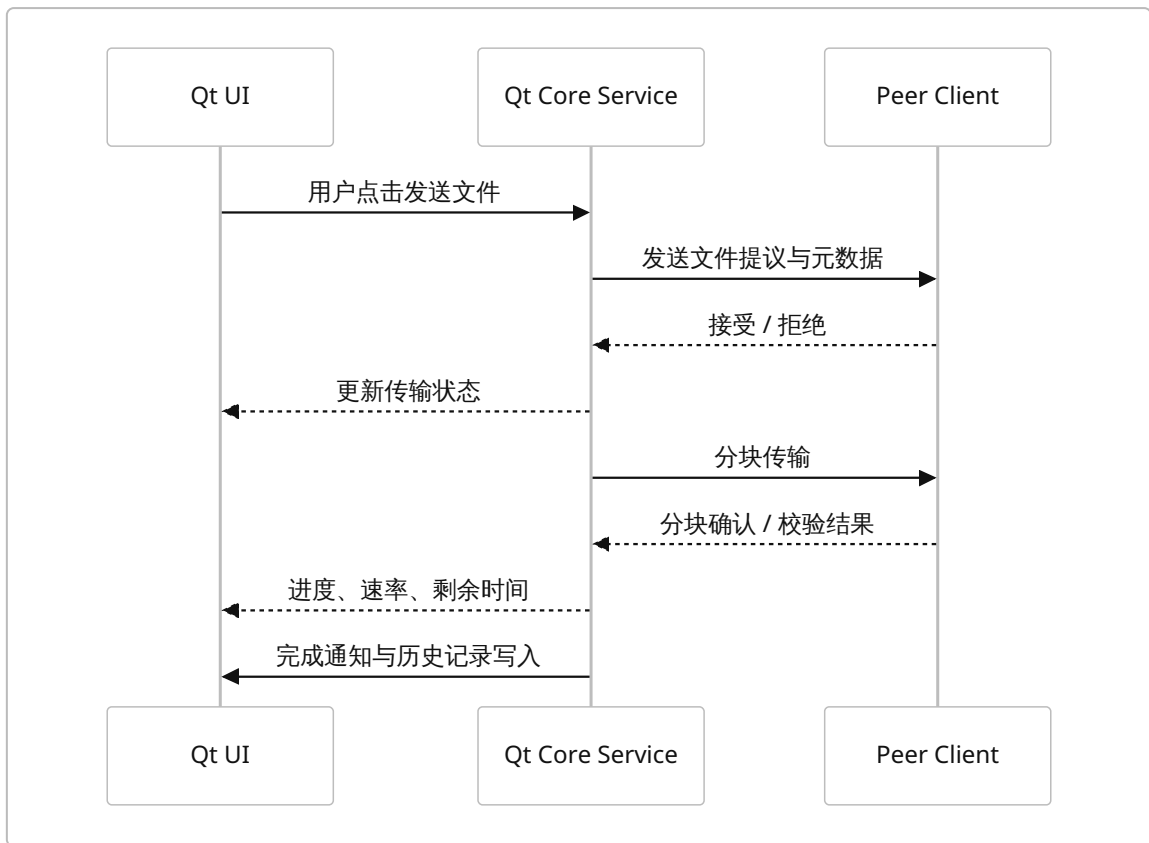
架构、构建与安全方案

推荐架构

MVP 我建议采用**单进程 Qt 客户端**：UI、局域网发现、消息收发、文件传输、共享目录浏览都在同一可执行文件内完成，但在代码结构上拆成独立模块：`ui`、`presence`、`message`、`transfer`、`share-index`、`storage`。这样你先拿到最快的可交付速度，同时未来如果要把文件索引/同步后台拆到 Rust，也不会推倒重来。



如果未来要拆后台服务，我建议是**Qt GUI + Rust 后台服务进程**，而不是立即引入 Tauri。对于同机 IPC，Qt 官方网络模块已经有成熟的 **QLocalServer / QLocalSocket**；如果你跨语言并且想做更“协议化”的边界，则可使用 **WebSocket** 或自己维护的 protobuf framing。虽然 Qt 也有 Qt GRPC，但由于 **Qt GRPC 在开源路线下属于 GPLv3 模块**，对潜在闭源商业化项目来说并不是默认优选。 7



IPC 与网络建议

如果 **MVP 不拆后台**，那最好的 IPC 就是**没有 IPC**。如果 **第二阶段拆成前后台**，我建议按下面顺序选：

场景	推荐	原因
同机 GUI 与后台	QLocalSocket / QLocalServer	Qt 官方原生、跨平台、最轻。 ⁴⁷
同机或本地回环、跨语言	WebSocket 或本地 HTTP	Qt 有成熟 WebSockets 模块，语言互操作简单。 ⁴⁸
强 schema、未来插件化	原生 gRPC 库	gRPC 官方支持 C++ / Go / Rust 等多语言。 ⁴⁹
LAN 发现	UDP 广播/组播	Qt Network 有 <code>QUdpSocket</code> 。 ⁵⁰
点对点文件传输	QTcpSocket + QSslSocket	TCP 稳定、TLS 现成，Qt 支持 TLS 1.3。 ⁵¹

构建与 CI/CD 建议

Qt 6 已把自身构建切到 **CMake + Ninja**，而 GitHub Actions 的 matrix 策略、缓存与 Docker Buildx 的多平台构建也都足够成熟。对你的项目，最务实的 CI/CD 组合是：**GitHub Actions 做触发和编排；Linux 发行版构建尽量容器化；Windows/macOS 走原生 runner；LoongArch 走自托管 runner。** ⁵²

推荐的发布管线如下：

平台	构建	打包	说明
Windows x86_64 / ARM64	原生 MSVC 2022	MSI 优先，工具上可选 CPack WiX/NSIS 或 Qt IFW	Windows 官方支持 ARM64；如要做 企业分发，MSI 更自然。 ⁵³
macOS Universal	原生 Xcode / Clang	.app + DMG	CPack 有 DragNDrop / productbuild；Qt 部署走 app bundle 模式。 ⁵⁴
Linux x86_64 / ARM64	容器化基线构 建	DEB + ApplImage	openKylin 明确是 DEB 体系； ApplImage 用来覆盖泛 Linux。 ⁵⁵
openKylin / 麒麟	原生 distro 包 / 自建 runner	DEB	把它当作发行版适配，而不是“神奇 万能 Linux”。 ⁵⁶

我建议你在仓库里固定三样东西：**Qt 版本**、**容器基线**、**编译器版本**。同时使用 `CMakePresets.json` 管理 profile，Linux 构建尽量在同一基础镜像里执行，以便提高可复现性。若未来需要 SBOM，Qt 官方从 **6.8** 开始已经提供第三方组件的 SBOM 文档。 ⁵⁷

安全与分发要求

这是一个**会监听局域网、会共享文件、会出现后台驻留**的应用，所以安全不是“后期再补”的问题。最关键的要求有五条。

第一，**默认最小暴露面**：如果你后来拆出本机后台服务，默认只绑定 **本地回环或本地 socket**；只有真正进行局域网发现与点对点传输时，才打开外部监听口。第二，**传输面加密**：Qt 的 `QSslSocket` 支持现代 TLS，包括 **TLS 1.3**，足以作为局域网点对点文件传输的加密基线。 ⁵⁸

第三，**签名与信誉**：Apple 官方要求面向 App Store 外分发的 macOS 软件做 **Developer ID 签名 + notarization**，这是 Gatekeeper 信任链的一部分；Microsoft 官方今天也明确把 **Azure Artifact Signing** 作为商店外 Windows 应用分发的推荐签名方案之一。 ⁵⁹

第四，**防火墙提示与入站规则**：Windows Defender Firewall 对监听型应用默认是阻断式模型，需要创建入站例外；macOS Application Firewall 也会影响入站连接。文件共享功能如果设计成“对局域网可见”，这一步就必须在安装器或首次运行向导中讲清楚。 ⁶⁰

第五，**如果你坚持用 Qt WebEngine**，则必须额外考虑 Linux 沙箱与内核能力；但正因为这些复杂度存在，我依然建议 **MVP 不用 WebEngine**。 ⁴²

MVP 实施计划与原型

推荐技术栈与实施计划

推荐栈：

- 语言：C++17
- UI：Qt 6.8 LTS，**Widgets-first**
- 网络：Qt Network，LAN 发现用 UDP，消息/传输用 TCP，传输加密用 TLS
- 存储：SQLite

- **构建**: CMake + Ninja
- **测试**: Qt Test
- **打包**: Windows MSI、macOS DMG、Linux DEB + ApplImage、openKylin 优先 DEB
- **第二阶段备选**: Rust 后台服务进程, IPC 用 QLocalSocket / WebSocket / 本地 HTTP

之所以是这个组合,是因为它同时满足了**原生桌面体验、最小认知复杂度、Linux/openKylin 落地性、以及后续可演进性**。Qt Creator 支持 GDB/CDB/LLDB 调试, Qt Test 可做 GUI/单元测试, QML 路线也有 QML Profiler 和设计器生态;而 Krita、OBS、Telegram Desktop 等大型桌面项目继续使用或迁移到 Qt6,也说明这条路线不是“小众个人癖好”,而是仍然处于主流可用区间。 ⁶¹

里程碑与验收标准

里程碑	交付物	成功指标
技术基线	Qt 6.8 LTS 工程脚手架; Windows/macOS/Linux 三端可编译; CI 跑通	三端冷启动成功; 版本信息、日志、崩溃回收可用
局域网发现与联系人	自动发现在线设备; 昵称、设备名、在线状态; 点对点会话列表	同一网段两台机器三秒内互相发现; 掉线三十秒内状态收敛
基础消息	文本消息收发; 未读计数; 本地历史	两端稳定连续发送一千条消息无崩溃; 历史可恢复
文件发送	选择文件发送; 进度、暂停、取消、失败重试	单文件—GB 传输成功; 异常断线后错误状态清晰
文件共享	共享目录、权限、浏览、下载	共享目录可被同网段其他设备浏览; 权限切换即时生效
托盘与后台	托盘菜单、最小化到托盘、开机自启可选	关闭主窗口后仍能后台驻留; 无托盘环境也能正常退出/恢复
打包发布	MSI、DMG、DEB、ApplImage; 签名与 notarization 流程	四种产物均可安装; 升级与卸载可用; 防火墙与权限提示清晰
第二阶段预埋	抽象传输引擎接口; 留好后台服务边界	把文件传输模块替换成本地 mock/service 不影响 UI 层调用

如果你非要先从 Qt5 起步

我依然不建议这样做,但若你有历史依赖、现成代码或某些内网平台强绑定 Qt5,那么**迁移策略**应该非常保守,尽量把将来的损失降到最低。Qt 官方“Porting to Qt 6”文档的第一条建议就是:**先把项目整理到 Qt 5.15**。同时,Qt 官方还提供了 **Qt 5 / Qt 6 CMake 兼容指南** 与 **Qt 5 Core Compat** 模块,目的就是帮助渐进迁移。 ⁶²

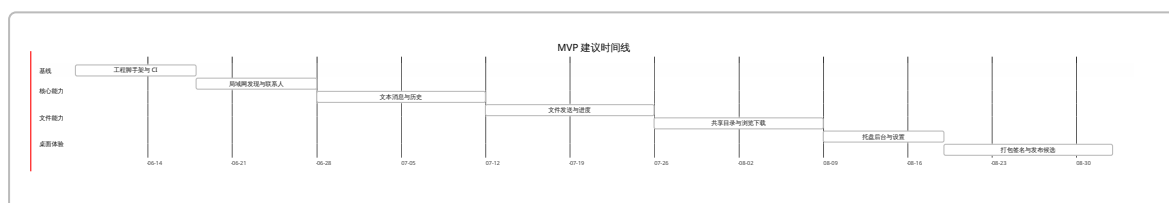
实操上应按下面做:

阶段	动作	迁移价值
先清理到 5.15	所有 deprecated API 清零	减少 Qt6 删除 API 的爆炸面
先改构建系统	把 qmake 迁到 CMake , 使用 Qt 5.15 的无版本 targets/commands	将来切 Qt6 只需改包名与少量 target 细节。 ⁶³

阶段	动作	迁移价值
避开敏感模块	不用 Qt WebEngine、Qt Extras、旧 QGL 类	这些在 Qt6 的 API/模块变化会更多。 ⁶⁴
控制 UI 技术债	Widgets 为主，QML 只做局部	降低图形后端与 QML API 细节带来的迁移成本
预埋边界	把网络与存储从 UI 中抽离	后续无论升 Qt6 还是拆 Rust 服务都更轻松

一句话概括：如果今天必须上 Qt5，也要用“像是在写 Qt6 时代的 Qt5 项目”这种方式来写。

时间线建议



低保真原型建议

主窗口

```

+-----+
| Logo 设备名: Lily-PC      在线: 12      搜索 [_____] [设置] |
+-----+
| 联系人 / 设备            | 会话: 设计组 |
+-----+
| ● 张三-PC                | [10:21] 张三: 你把文档共享出来了吗? |
| ● 财务机-01              | [10:22] 我: 已经开了 /ProjectShare |
| ○ 测试机-Mac             | [10:23] 张三: 我看到了, 准备下载 |
| ● 会议室屏幕              | |
+-----+
| [新建会话] [群组]        | 输入消息... |
| [共享文件] [传文件]      | [_____] [发送] |
+-----+
| [A] 在线状态 [B] 共享入口 [C] 设备搜索 [D] 会话区要支持拖拽发文件 |
+-----+

```

文件共享页

```

+-----+
| 文件共享 |
+-----+
| 共享状态: [已开启] 对谁可见: [同网段所有人 v] 发现方式: [自动] |
| 共享目录列表 |
| [✓] D:\ProjectShare 权限: 只读 大小: 23.4 GB [浏览] [移除] |
+-----+

```

```

| [✓] C:\PublicDrop      权限: 可上传    大小: 2.1 GB      [浏览] [移除]
| [添加目录] [刷新索引] [暂停共享]
+-----+
| 当前在线访问者
| 张三-PC      正在浏览 /ProjectShare      最近活动: 10 秒前
| 测试机-Mac   正在下载 spec.docx      速率: 12 MB/s
+-----+
| [A] 权限必须目录级可配  [B] 要显示在线访问者  [C] 要有一键暂停共享
+-----+

```

传输中心

```

+-----+
| 传输中心
+-----+
| 类型  文件名          对端      状态    进度    速率    操作
| 发送  架构说明.pdf      张三-PC   进行中  67%     18MB/s  [暂停]
| 接收  logo.png           测试机-Mac 已完成  100%    -       [打开]
| 发送  build.zip        财务机-01 失败    12%     0       [重试]
+-----+
| 详细信息
| 校验方式: SHA-256      断点续传: 已开启      存储位置: D:\Downloads
+-----+
| [A] 失败必须可重试  [B] 要有校验信息  [C] 进度、速率、剩余时间必须一眼可见
+-----+

```

首次启动向导

```

+-----+
| 欢迎使用 LAN IM
+-----+
| 第一步: 设备身份
| 设备显示名 [ Lily-PC_____ ]
| 头像      [ 选择 ]
+-----+
| 第二步: 网络与发现
| [✓] 自动发现同网段设备
| [ ] 仅手动添加可信设备
| [✓] 启动时最小化到托盘
+-----+
| 第三步: 共享设置
| 默认下载目录 [ D:\Downloads_____ ] [浏览]
| 默认共享目录 [ _____ ] [添加]
+-----+
| [上一步]                                [完成并启动]
+-----+
| [A] 首次安装时就把发现、托盘、共享目录讲清楚, 可减少后续客服成本
+-----+

```

建议下一步

如果你现在就要进场开发，我建议按这个顺序执行：**先做技术脚手架和 Windows/Linux 双端冷启动，再做 LAN 发现，再做文本消息，再做文件发送，最后才上共享目录浏览与打包签名**。原因不是功能重要性，而是**发现机制、传输机制、打包机制**是这类产品最容易在跨平台时出大坑的三件事，越早验证越值钱。

风险与缓解

风险	影响	缓解
继续从 Qt5 启动	一开始就背上 EoS 技术债	直接上 Qt 6.8 LTS
一开始就上 QML + WebEngine	Linux/openKylin/LoongArch 复杂度陡升	Widgets-first, MVP 不上 WebEngine
一开始就做 Qt + Rust/Go 双进程	MVP 节奏变慢，调试成本翻倍	先纯 C++；第二阶段再拆后台
误用 Qt GRPC	可能踩到 GPLv3 模块许可问题	开源 LGPL 路线下优先用 QLocalSocket / WebSocket / 原生 gRPC 库
把麒麟当成“普通 Ubuntu”	打包与图形栈兼容不稳定	以 openKylin DEB 规范和实机回归为准
托盘依赖过重	Wayland / UKUI / GNOME 差异导致功能缺失	设计无托盘回退路径
过早追求全平台多架构齐发	发布节奏被拖垮	第一发布线只保 x86_64；ARM64 第二优先；LoongArch 作为实验线
忽略签名与防火墙	用户安装即被阻断或不信任	把签名、notarization、防火墙解释写进发布流程和首次启动向导

开放问题与局限

本报告基于官方资料可以高置信度地判断 **Qt6 胜过 Qt5、Widgets-first 胜过 Tauri 首发、以及 openKylin/LoongArch 需自维护**；但有两类问题仍然需要你在立项后用原型实测确认：其一，**UKUI/麒麟系对托盘、通知、开机自启与后台驻留的具体行为差异**；其二，**你真实目标机器上的 glibc、图形栈、内核 user namespace 设置是否会影响 Linux 打包与运行**。这两项不适合继续停留在文档推断层面，应该通过最早一轮技术验证来关闭。 ⁶⁵

¹ ⁹ ³⁰ **Commercial LTS Qt 5.15.19 Released**
https://www.qt.io/blog/commercial-lts-qt-5.15.19-released?utm_source=chatgpt.com

² ⁸ ¹¹ ³¹ **Qt Releases | Qt 6.11**
https://doc.qt.io/qt-6/qt-releases.html?utm_source=chatgpt.com

³ ⁷ <https://doc.qt.io/qt-6/ipc-overview.html>
<https://doc.qt.io/qt-6/ipc-overview.html>

⁴ ³⁷ <https://v2.tauri.app/start/prerequisites/>
<https://v2.tauri.app/start/prerequisites/>

- 5 Qt Qml Tooling
https://doc.qt.io/qt-6/qtqml-tooling.html?utm_source=chatgpt.com
- 6 25 <https://doc.qt.io/qt-6/qtwebengine-index.html>
<https://doc.qt.io/qt-6/qtwebengine-index.html>
- 10 16 18 38 Supported Platforms | Qt 6.11
https://doc.qt.io/qt-6/supported-platforms.html?utm_source=chatgpt.com
- 12 63 <https://doc.qt.io/qt-6.8/zh/cmake-qt5-and-qt6-compatibility.html>
<https://doc.qt.io/qt-6.8/zh/cmake-qt5-and-qt6-compatibility.html>
- 13 26 27 52 57 <https://doc.qt.io/qt-6/qt6-buildsystem.html>
<https://doc.qt.io/qt-6/qt6-buildsystem.html>
- 14 20 <https://www.docs4dev.com/docs/qt/5.15/supported-platforms.html>
<https://www.docs4dev.com/docs/qt/5.15/supported-platforms.html>
- 15 45 53 <https://doc.qt.io/qt-6/windows.html>
<https://doc.qt.io/qt-6/windows.html>
- 17 39 <https://doc.qt.io/qt-6/linux.html>
<https://doc.qt.io/qt-6/linux.html>
- 19 ISO download-openKylin 开放麒麟社区官网 | 开源聚力
https://www.openkylin.top/downloads/os-en.html?utm_source=chatgpt.com
- 21 <https://cmake.org/cmake/help/book/mastering-cmake/chapter/Packaging%20With%20CPack.html?highlight=zip>
<https://cmake.org/cmake/help/book/mastering-cmake/chapter/Packaging%20With%20CPack.html?highlight=zip>
- 22 Qt WebEngine Overview
https://doc.qt.io/qt-6/qtwebengine-overview.html?utm_source=chatgpt.com
- 23 24 Introducing Qt Quick 3D: A high-level 3D API for Qt Quick
https://www.qt.io/blog/2019/08/14/introducing-qt-quick-3d-high-level-3d-api-qt-quick?utm_source=chatgpt.com
- 28 <https://doc.qt.io/qt-6/licensing.html>
<https://doc.qt.io/qt-6/licensing.html>
- 29 <https://krita.org/en/release-notes/krita-5-3-release-notes/>
<https://krita.org/en/release-notes/krita-5-3-release-notes/>
- 32 36 <https://v2.tauri.app/blog/tauri-20/>
<https://v2.tauri.app/blog/tauri-20/>
- 33 61 <https://doc.qt.io/qt-6/qtest-overview.html>
<https://doc.qt.io/qt-6/qtest-overview.html>
- 34 49 <https://grpc.io/docs/languages/>
<https://grpc.io/docs/languages/>
- 35 <https://grpc.io/docs/languages/go/quickstart/>
<https://grpc.io/docs/languages/go/quickstart/>
- 40 <https://doc.qt.io/qtforpython-6/overviews/qtwidgets-application-windows.html>
<https://doc.qt.io/qtforpython-6/overviews/qtwidgets-application-windows.html>
- 41 65 <https://doc.qt.io/qt-6/qsystemtrayicon.html>
<https://doc.qt.io/qt-6/qsystemtrayicon.html>

- 42 <https://doc.qt.io/qt-6/qtwebengine-platform-notes.html>
<https://doc.qt.io/qt-6/qtwebengine-platform-notes.html>
- 43 <https://docs.appimage.org/reference/best-practices.html>
<https://docs.appimage.org/reference/best-practices.html>
- 44 <https://doc.qt.io/qt-6/cross-compiling-qt.html>
<https://doc.qt.io/qt-6/cross-compiling-qt.html>
- 46 <https://doc.qt.io/qt-6/qtprocessordetection.html>
<https://doc.qt.io/qt-6/qtprocessordetection.html>
- 47 <https://doc.qt.io/qt-6/qlocalserver.html>
<https://doc.qt.io/qt-6/qlocalserver.html>
- 48 <https://doc.qt.io/qt-6/qtwebsockets-index.html>
<https://doc.qt.io/qt-6/qtwebsockets-index.html>
- 50 <https://doc.qt.io/qt-6/qudpsocket.html>
<https://doc.qt.io/qt-6/qudpsocket.html>
- 51 <https://doc.qt.io/qt-6/qtcpsocket.html>
<https://doc.qt.io/qt-6/qtcpsocket.html>
- 54 <https://doc.qt.io/qt-6/macos.html>
<https://doc.qt.io/qt-6/macos.html>
- 55 56 openKylin打包指南
https://docs.openkylin.top/zh/04_%E7%A4%BE%E5%8C%BA%E8%B4%A1%E7%8C%AE/%E5%BC%80%E5%8F%91%E6%8C%87%E5%8D%97/openKylin%E6%89%93%E5%8C%85%E6%8C%87%E5%8D%97?utm_source=chatgpt.com
- 58 <https://doc.qt.io/qt-6/qsslsocket.html>
<https://doc.qt.io/qt-6/qsslsocket.html>
- 59 <https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution>
<https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution>
- 60 <https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/rules>
<https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/rules>
- 62 <https://doc.qt.io/qt-6/portingguide.html>
<https://doc.qt.io/qt-6/portingguide.html>
- 64 <https://doc.qt.io/qt-6/opengl-changes-qt6.html>
<https://doc.qt.io/qt-6/opengl-changes-qt6.html>